



Garden Guard Final Project Report

Authors & Emails:

Mohammad Shaheer Siddiqi - mss425@scarletmail.rutgers.edu - Project Lead

Abdul Kader - ajk295@scarletmail.rutgers.edu - Project Implementer & Evaluator

Daniyal Siddiqui - dfs107@scarletmail.rutgers.edu - Project Report Writer

Zhouyang Chen - zc399@scarletmail.rutgers.edu - Project Implementer

Course: INTRO ARTIFIC INTELL 01:198:440:05

Instructor: Casimir A. Kulikowski

Submission Date: December 18, 2025

1. Overview

Our report contains how we designed and tested GardenGuard, our AI detection app that looks at plant leaves to see if they are healthy or sick. If the AI does deem the plant as sick, it also predicts exactly what disease it has and how to remedy it. Our main purpose was to create a fast and free diagnostic tool for beginner gardeners by combining a powerful AI model with an easy-to-use app interface.

The bulk of our project involved pre-training a Convolutional Neural Network, which is specially designed for image recognition, and "fine-tuning" it especially to recognize specific plants and their diseases. This report will focus on why we chose this specific AI model, how well it worked on different types of plants, and how our results compared to other scientific studies.

2. Rationale and Goals

Plant diseases kill crops and ruin gardens, so it is essential to prune them before they take root. But however, spotting them early on is hard because most people aren't plant experts. We wanted to build a lightweight tool that acts like an expert you can keep in your pocket. While big research papers show AI can be very accurate in labs with expensive hardware, we wanted to see if we could build a version that can work just as efficiently on the hardware you can find in your smartphone.

Our specific goals were to:

1. Build an image classifier using Transfer Learning where we teach an existing AI new skills to identify both the plant type and the disease.
2. Calculate the accuracy of the model, not just an overall measure, but between specific plants as well.
3. Compare our results to the major studies we referenced to see where we stand in terms of how efficient our model was.

4. Deliver a working demonstration that doesn't just guess, but tells the user how confident the AI believes it is in its answer.

3. Materials and Methods

3.1 Data source

We used the PlantVillage dataset, which is the gold standard for this kind of research. It contains over 54,000 images of healthy and diseased leaves. We accessed this via Kaggle. The images are taken in mostly controlled settings like gray backgrounds, which makes it great for benchmarking our AI's raw ability to learn patterns.

3.2 Dataset

To make sure our AI wasn't cheating by memorizing images, we split the data into three separate sets: a training set to teach it, a validation set to check its progress while learning, and a testing set it had never seen before.

Table 1. Dataset composition by plant type

Plant type	Classes included	Train (80%)	Val (10%)	Test (10%)
Apple	4 (Scab, Rot, Rust, Healthy)	2,537	317	317
Corn (Maize)	4 (Blight, Rust, Spot, Healthy)	3,082	385	385
Grape	4 (Rot, Measles, Blight, Healthy)	3,249	406	407
Potato	3 (Early Blight, Late Blight, Healthy)	1,722	215	215

Tomato	10 (Mosaic Virus, Spot, Mites, etc.)	14,529	1,816	1,816
Total (All)	38 Classes	43,444	5,430	5,431

3.3 Image preprocessing and augmentation

Before feeding the images to the AI, we resized them all to 224x224 pixels. This is the standard size our AI model works with. We also normalized the pixel colors so the math inside the neural network can work smoothly.

To make the model tougher, we used data augmentation. We randomly flipped images, rotated them slightly, or zoomed in them during training. This forces the AI to look for the disease patterns, and not just memorize that, for example, a leaf in the top left corner always means it is an apple.

3.4 Choice of image classifier (The AI Model)

We used a technique called Transfer Learning. Instead of building a brain from scratch which takes weeks and millions of images, we started with MobileNetV2.

Why MobileNetV2?

We chose this specific Convolutional Neural Network because it is optimized for mobile phones.

1. **Efficiency:** It uses Depthwise Separable Convolutions. In layman's terms, this is a math technique that splits the heavy calculation into two smaller steps, making the model run 10x faster than older models like VGG16 without losing much accuracy.
2. **Pre-training:** It was already trained on ImageNet which is a massive database of everyday objects. It already knows how to discern edges, curves, and textures before we even showed it a leaf.

Baseline: We compared this against a standard ResNet-50 to see if the lighter mobile model lost too much accuracy.

3.5 Training procedure

We used the Adam optimizer (the standard algorithm for updating the AI's brain) and categorical cross-entropy to measure error. We trained for 20 epochs or rounds, but used "Early Stopping" to stop automatically if the model stopped improving, to prevent overfitting.

Table 2. Training configuration

Training setting	Value
Input size	224 x 224 pixels
Optimizer	Adam
Learning Rate	0.0001 (1e-4)
Batch size	32
Epochs	20
Loss Function	Categorical Cross-Entropy
Environment	TensorFlow/Keras on Google Colab (T4 GPU)

3.6 Implementation

We built the pipeline using Python and TensorFlow/Keras. The workflow is simple: the user uploads an image, the system preprocesses it to 224x224, runs it through the MobileNetV2 model, and the model outputs a list of probabilities (e.g., "98% confident this is Apple Scab").

3.7 Roles

- Mohammad (Lead): Selected the MobileNetV2 architecture and managed the project scope pivot.
- Abdul & Zhouyang (Implementers): Wrote the Python scripts for data augmentation and training the model.
- Daniyal (Writer): Analyzed the test results and wrote the final evaluation report.

4. Results

4.1 Evaluation metrics

We looked at the accuracy of overall correct answers and the Macro F1 score, an average to make sure we aren't just ignoring rare diseases.

4.2 Overall performance

On the final test set, our MobileNetV2 model performed impressively well, though slightly lower than the heavier ResNet baseline.

Table 3. Overall test performance

Model	Accuracy	Macro F1	Size (MB)
Baseline: ResNet-50	96.5%	0.958	~98 MB
Our Model: MobileNetV2	94.2%	0.931	~14 MB

Conclusion: Our MobileNetV2 model is 7 times smaller than the baseline but only lost 2.3% accuracy. This is more than enough to prove it is the right choice for a mobile app.

4.3 Plant-type dependence

We were asked to check if some plants are harder to diagnose than others. The answer is yes.

Table 4. Plant-type dependent performance

Plant type	Test images	Accuracy	Major Confusion
Apple	317	97.8%	Very distinct; rarely confused.
Corn	385	98.4%	Distinct linear leaf structure makes it easy.

Grape	407	95.1%	Black Rot sometimes confused with Measles.
Potato	215	88.2%	Confused with Tomato Early Blight.
Tomato	1,816	91.5%	Confused with Potato Early Blight.

Key Finding: Potato and tomato are biologically related as they are both from the Solanaceae family, and their "Early Blight" disease looks almost identical. The AI struggled to tell the difference between a sick potato leaf and a sick tomato leaf because the fungus looks the same on both.

4.4 Error analysis

We found that errors mostly came from:

1. Inter-disease similarity: As mentioned, tomato and potato diseases look very alike.
2. Background Noise: If a leaf was photographed against a messy background like dirt or grass instead of the gray background the model was used to, confidence dropped.
3. Early Stages: Very mild disease spots were sometimes labeled as "Healthy" by the AI.

4.5 Comparison with prior work

We compared our work to two famous studies: Mohanty et al. (2016) and Ferentinos (2018).

Table 5. Comparison with peer-reviewed benchmarks

Study	Dataset	Model	Accuracy	Note
Mohanty et al. (2016)	PlantVillage	GoogLeNet	99.35%	Used a heavy server-side model.
Ferentinos (2018)	Open DB	VGG16	99.53%	Very slow inference speed.

This Project	PlantVillage	MobileNetV2	94.20%	Optimized for speed/mobile use.
---------------------	--------------	--------------------	---------------	--

Interpretation: Our accuracy is lower (~94% vs 99%), but this is intentional. Mohanty and Ferentinos used massive models that required powerful servers. We used a lightweight model designed for phones. We traded a small amount of accuracy for a huge gain in speed and portability.

5. Conclusions

Our experiments prove that Transfer Learning is a viable strategy for plant disease detection. By using MobileNetV2, we achieved more than 94% accuracy with a model small enough to run on a smartphone. The critical AI method used was the Convolutional Neural Network, which automatically learned to detect lesion textures without us needing to program them manually.

6. Lessons Learned and What We Would Do Differently

Lessons:

- **Data is essential:** We learned that the AI is only as good as the photos. If the lighting is bad, the prediction is bad.
- **Class Imbalance:** We noticed we had way more Tomato images than Potato images, which is why the Potato accuracy was lower, because the AI practiced less on it.

What we would change:

If we did this again, we would:

1. **Collect field data:** Test the model on real photos from a garden, and not just the lab photos from the dataset.
2. **Object Detection:** Instead of just classifying the whole image, we would draw a box around the specific sick leaf it was focusing on.

7. Individual Statement of Learning

- **Mohammad:** "I learned how to evaluate trade-offs in AI. Choosing MobileNetV2 meant sacrificing 2% accuracy, but it made the app 7x smaller. Understanding that balance was a key lesson."
- **Abdul:** "I learned the practical side of Deep Learning. For example, writing the code to augment images, like flipping and rotating them, showed me that this is how we prevent the AI from just memorizing the training data."

- **Daniyal:** "I learned how to analyze the Confusion Matrix. Seeing exactly *why* the model confused Potato and Tomato leaves helped me understand that AI sees textures, not biological families."
- **Zhouyang:** "I learned about the training loop. Watching the validation loss drop while the training loss dropped told me our model was actually learning and not just guessing."

8. References

1. **Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).** Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419.
2. **Ferentinos, K. P. (2018).** Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311-318.
3. **Hughes, D. P., & Salathé, M. (2015).** An open access repository of images on plant health. arXiv:1511.08060.
4. **Kaggle.** PlantVillage Dataset. Retrieved 2025.

9. Appendix: Detailed Results

Appendix A: Per-class Classification Report (Snippet)

This table shows the detailed scores for our most difficult classes.

Class Name	Precision	Recall	F1-Score	Support (N)
Apple___Apple_scab	0.96	0.98	0.97	63
Apple___Healthy	0.99	0.99	0.99	164
Potato___Early_blight	0.85	0.82	0.83	100
Potato___Healthy	0.98	0.95	0.96	15
Tomato___Early_blight	0.88	0.89	0.88	100

Tomato___Healthy	0.97	0.98	0.97	159
------------------	------	------	------	-----

Appendix B: Confusion Matrix Analysis

This data explains the potato/tomato confusion mentioned in the report.

Confusion between Potato and Tomato Early Blight:

- Out of 100 Potato Early Blight test images:
 - 82 were correctly identified.
 - 12 were misclassified as Tomato Early Blight.
 - 6 were misclassified as Late Blight.
- Analysis: The visual similarity of the *Alternaria solani* fungus on both plants is extremely high, which causes the AI to confuse the plant even if it correctly identified the disease type.

Appendix C: Qualitative Examples

1. Success: A photo of a Corn leaf with Common Rust. The AI predicted "Corn Common Rust" with 99.8% confidence. The distinct orange pustules were clearly picked up by the model.
2. Failure: A photo of a Potato leaf with Early Blight. The AI predicted "Tomato Early Blight" with 65% confidence. The leaf shape was obscured by the close-up zoom, causing the model to guess the more common plant (Tomato).